

Privacy as a Release Property: Proof-Carrying Anonymity Receipts for Anonymous DHTs

Draft v0.53 (February 2026; archive v100)

Abstract

Anonymous DHT designs are typically argued “on paper” (protocol structure, asymptotic bounds) while deployments leak through control plane details, retries, caches, and operational drift. We propose a complementary stance: treat anonymity as a *release property* that can be independently verified at publish time.

We present a practical, proof-carrying workflow based on (i) replayable release bundles that commit to evidence artifacts by digest, (ii) a claims ledger that maps each privacy claim to its evidence, (iii) a compact Privacy Verification Summary Attestation (PVSA) for downstream consumers, and (iv) policy-as-code gates that can block releases when budgets regress. The output is an *anonymity receipt*: a signed, checkable statement of privacy budgets and assumptions that downstream users and auditors can verify offline. We keep the paper brief by importing the underlying theory modules (stop-time padding, retry amplification, destination privacy, congestion witnesses) from companion papers in this series.

Scope. We treat anonymity as a *release property*: how to publish privacy claims with a replayable evidence surface that third parties can verify offline and track over time.

Imports. We import the underlying budget semantics from the companion papers (odds inflation, retry amplification, destination privacy, congestion witnesses), and treat ABOM/OINL as the canonical packaging of claim inputs.[4, 3] We also treat beacon/VRF binding and append-only publication as imported primitives (Synthesis 9).[22] Exposure-normal-form identity and receipt-line-item semantics are imported from the synthesis notes on trace interfaces and receipt schemas.[20, 21] The present paper owns only the *receipt/ledger* workflow and its anti-equivocation threat model.

Exports. A release-facing receipt stack: (i) release bundle manifest format, (ii) claims ledger entries, (iii) a signed privacy receipt envelope, and (iv) the compact PVSA summary attestation for dashboards and dependency tooling.

Artifact policy. Source-only; supporting bundle formats, scripts, and example receipts are available on request.[1]

1 Introduction

Distributed hash tables (DHTs) are a common substrate for decentralized systems, but dominant DHT designs were not built to hide *who asked what, when* [10]. Modern deployments add delegated routing aids, caches, and operational behaviors that dominate leakage in practice. Large-scale

passive monitoring studies in deployed ecosystems (e.g., IPFS) underline that control planes are themselves privacy-critical [11].

This series develops modular anonymity bounds for Anonymous DHT lookups. This paper asks the release-engineering question: *How can a deployment publish a privacy claim that third parties can mechanically verify, audit over time, and compare across releases?*

1.1 Privacy as a release property

The core idea is to treat privacy claims like supply-chain claims. A release should ship a signed, replayable evidence package: a verifier need not possess every private artifact, but should be able to re-check digests, signatures, and exported budgets. Artifacts may be distributed “on request,” while the release carries:

- a *manifest* committing to evidence artifacts by digest;
- a *claims ledger* linking each high-level statement to its evidence;
- a compact summary attestation (PVSA) suitable for dashboards and dependency tooling; and
- a *policy gate* that declares acceptable budgets/assumptions and fails closed.

2 Threat model and failure modes

We assume an adversary who can observe some subset of overlay messages and can probe repeatedly. The release workflow must additionally handle a different adversary: a publisher who misreports evidence, cherry-picks outputs, or equivocates by giving different claims to different audiences.

Definition 1 (Release property). *A privacy claim is a release property if it is expressed as a signed statement whose validity can be checked by an offline verifier against a manifest of evidence digests and against a public policy, without trusting the publisher beyond the signing keys and transparency receipts.*

Equivocation taxonomy. Separating statement-level forks (issuer equivocation) from log-level split views (transparency equivocation) matters because it determines what monitors can publish and who is blameable. Table 1 is the minimal taxonomy we use throughout the series.

3 Objects: bundles, ledgers, receipts

3.1 Release bundle

A *release bundle* is a directory (or archive) containing:

1. `manifest.json`: file list with digests;
2. evidence artifacts (reports, plots, tables) referenced by claims;
3. `attest.json`: a signature over the manifest digest; and
4. optional derived objects (e.g., transparency receipts, PVSA summaries).

The manifest provides a stable root of trust: a verifier checks that every referenced evidence object matches its digest.

Failure / attack	What diverges?	Public evidence object	Primary blame
Issuer equivocation (policy fork)	Two <i>accepted</i> privacy statements extend the same prev head but differ in payload (budgets, evidence digests).	Pair of signed statement envelopes + receipts; fork detected by scanning children of a prev digest.	Issuer / release authority
Log split-view (inconsistent checkpoint)	Two signed checkpoints with the same tree_size but different roots are shown to different clients.	Pair of signed checkpoints; optionally augmented with failed consistency proofs.	Log operator (or compromised log key)
Witness equivocation	A witness signs checkpoints on two inconsistent histories.	Two witness cosignatures over inconsistent checkpoints for the same log id.	Witness operator
Monitor failure (no one is watching)	Divergence exists but is not compared/gossiped.	Absence of evidence; mitigated by explicit monitoring SLAs and cross-operator gossip.	Governance / relying party

Table 1: Equivocation taxonomy for transparency-logged privacy receipts.

3.2 Claims ledger

A claims ledger maps each high-level statement to one or more evidence artifacts (by digest). It is deliberately explicit. The point is not to publish every private artifact in public; the point is to make claims *addressable* so that an auditor can request exactly the items needed to validate a line item.

3.3 Receipt and PVSA

A *privacy receipt* is a signed statement of budgets and assumptions. A PVSA is a compact, machine-readable summary extracted from the receipt: headline $\varepsilon(\delta)$ values, modes, key assumptions, and the digests that bind the claim to evidence. A receipt may optionally carry transparency receipts (Merkle inclusion/consistency) so that equivocation becomes detectable (§5).

3.4 Claim lineage in the ledger

A release ledger should not compare budgets until it first declares *what kind of change occurred*. The important distinction is not merely “old receipt versus new receipt” but whether the new statement preserves the old exposure normal form. A ledger entry should therefore carry a predecessor pointer and a short relation tag such as **same-enf**, **coarsening**, or **new-interface**. [20, 21, 5]

Proposition 1 (When a release may reuse a claim lineage). *Consider two consecutive release statements for the same deployment. A verifier may treat the later statement as belonging to the same claim lineage without full re-issuance only if either (i) the exposure normal form, repetition unit, and public conditioning are unchanged, or (ii) the later statement declares an explicit coarsening of the earlier interface under the same repetition unit and public conditioning. Any other change*

Mode	ε_{old} @ $\delta = 10^{-6}$	ε_{new}	$\Delta\varepsilon$	ε_{old} @ $\delta = 10^{-9}$	ε_{new}	$\Delta\varepsilon$	Policy
conservative	1.151	1.301	+0.150	1.613	1.763	+0.150	fail
dag_trusted	1.031	1.181	+0.150	1.492	1.642	+0.150	fail

Table 2: Example PBOM diff (privacy patch notes). Positive $\Delta\varepsilon$ indicates worse privacy; policy-as-code can fail closed.

should be published as a new interface claim, with a fresh claim object and an explicit predecessor link in the ledger.

Proof sketch. Release comparison is meaningful only when the two statements speak about the same experiment. Case (i) preserves the experiment; Case (ii) preserves it up to data processing and is therefore conservative. Other interface changes alter what is being certified or how repetitions compose, so a budget delta by itself is no longer an apples-to-apples release comparison. $\square$

3.5 PBOM and ABOM integration

We use *ABOM/OINL* as the canonical “what does this privacy claim mean?” packaging layer [2]. At release time: (1) the ABOM identifies the schema versions used by receipts; and (2) a compact PBOM diff becomes “privacy patch notes,” showing budget deltas across releases. An example PBOM diff is in Table 2.

4 Verification and policy gates

4.1 Minimal verifier pipeline

A conservative verifier should:

1. validate the manifest and signature chain (bundle integrity);
2. validate the receipt schema and canonicalize JSON prior to hashing/signing (e.g., JCS) [14];
3. replay the accountant arithmetic for each claimed module (retries, stop-time padding, destination equalization, congestion witnesses); and
4. check that headline PVSA values match recomputation.

The key property is replay: a receipt should be the output of deterministic scripts over a pinned evidence set. Notarized evaluation certificates ([6]) are one way to make replay credible without publishing all raw traces.

4.2 Fail-closed operator checklist

Table 3 is an opinionated checklist: what to export, what to replay, and what to gate in CI. Several line items correspond directly to companion papers in this archive (schedule-only retries [8], congestion receipts [9], stealth audits [7]).

Channel	Witness artifact	Gate / verification step	What it prevents
Witness integrity	<code>manifest.json</code> + <code>attest.json</code>	Manifest validation + signature verify	Stops unsigned edits / supply-chain drift.
Receipt governance	Receipt + verification report	t -of- k auditor policy; fail closed on policy mismatch	Stops policy drift and issuer equivocation.
Transparency API	Entry locator + receipt	Key discovery + receipt resolution (e.g., SCRAPI) [19]	Stops unverifiable transparency claims / stale keys.
Accounting frontier	Budget curves / line items	Replay accountants; diff in CI	Stops silent budget regressions.
Retries / aborts	Retry transcript + policy	Check retry contract; schedule-only retries [8]	Stops many-testing amplification.
Destination privacy	MC-EQ / MUCC parameters	Mixing / cover checks; tail-mass bounds	Stops key-dependent routing signatures.
Congestion leakage	Congestion certificate + receipts	Verify Congestion-EQ accountant; log receipts [9]	Stops unverifiable congestion-budget claims.

Table 3: Fail-closed operator checklist: turn narrative privacy promises into explicit checks that can block releases.

5 Transparency anchoring

Receipts reduce equivocation by anchoring statements in append-only logs. The mechanics follow Certificate Transparency-style inclusion/consistency proofs [15]. SCITT generalizes this pattern to broader supply-chain statements, including reference APIs for statement submission and receipt retrieval [18, 19].

We recommend separating two layers:

- **Statement layer:** what was signed and what budgets were claimed (issuer equivocation lives here).
- **Log layer:** what the transparency service committed to (split views live here).

The equivocation taxonomy in Table 1 tells auditors what evidence object to publish when things go wrong.

6 How theory modules appear as receipt line items

A receipt is only as good as its line items. Rather than re-stating per-module semantics here, we treat the minimal line-item tuple schema as owned by the receipt-schema synthesis note (Synthesis 12).[21] Mechanism papers should export their knobs/accountants so they can populate the schema fields (exposure id, witness tier, threat window, usage assumptions, observation model, budget value, and lineage relation), and this paper focuses on publication: transparency anchoring, equivocation mitigation, and change control.

7 Related work and context

The workflow is inspired by supply-chain verification frameworks and transparency logs. In-toto treats a software build pipeline as a signed set of steps whose integrity can be verified end-to-end [12]. Sigstore lowers the operational burden of signing and leverages transparency logs to make signing activity auditable [13]. SLSA defines incrementally stronger supply-chain assurances and encourages verifiable provenance [16]. SPDX is a widely used SBOM communication format [17]. We use these as analogies: our receipts are a “privacy SBOM,” and ABOM/OINL is the canonical packaging layer [2].

8 Conclusion

Privacy claims for anonymous overlays should be publishable artifacts with change control, not one-off arguments. Treating anonymity as a release property turns budgets into receipts that can be verified, monitored, and compared across releases. The series’ core research question becomes operational: *what do we publish, what do we keep private, and what can third parties still check?*

9 Takeaways

- **Treat privacy as a release property:** budgets are versioned artifacts with change control, not one-off arguments.
- **Receipts are replayable:** a receipt should be deterministically reproducible from a pinned evidence set, ideally under notarization when raw traces are not published.
- **Fail closed:** publish explicit gates (manifest validation, retry-policy checks, budget replays, transparency receipts) so regressions block releases rather than shipping silently.
- **Where to cite:** use ABOM/OINL for claim packaging and W-Congestion-EQ for congestion receipts; Operational A provides retry contracts.

References

- [1] Anonymous. *Artifact and Disclosure Policy for the One-True-Archive (Source-Only Public Release)*. Synthesis 6, The-One-True-Archive (2026).
- [2] Anonymous. *ABOM and OINL: An Anonymity Bill of Materials and an Odds-Inflation Nutrition Label*. Anonymity series Paper C, The-One-True-Archive (2026).
- [3] Anonymous. *ABOM and OINL for Anonymity Claims*. Anonymity series Paper C, The-One-True-Archive (2026).
- [4] Anonymous. *Odds Inflation, Privacy Loss, and Endpoint Anonymity Bounds*. Anonymity series Paper A, The-One-True-Archive (2026).
- [5] Anonymous. *Disagreement-Gated Recertification: Change Control for Certified Anonymous Systems*. Certified series Paper C, The-One-True-Archive (2026).
- [6] Anonymous. *Notarized Anonymity Certificates*. Evaluation series Paper 1, The-One-True-Archive (2026).
- [7] Anonymous. *Advantage Contracts and Stealth Audits*. Evaluation series Paper 2, The-One-True-Archive (2026).

- [8] Anonymous. *Schedule-Only Retries for Anonymous DHT Lookups*. Operational series Paper A, The-One-True-Archive (2026).
- [9] Anonymous. *W-Congestion-EQ: Verifiable Congestion Budgets (Certificates + Receipts)*. Congestion series Paper 3, The-One-True-Archive (2026).
- [10] P. Maymounkov and D. Mazières. Kademlia: A peer-to-peer information system based on the XOR metric. In *IPTPS 2002*, LNCS 2429, 2002.
- [11] L. Balduf, S. Henningsen, M. Florian, S. Rust, and B. Scheuermann. Monitoring data requests in decentralized data storage systems: A case study of IPFS. arXiv:2104.09202, 2021.
- [12] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *USENIX Security*, 2019.
- [13] Z. Newman, J. Speed Meyers, and S. Torres-Arias. Sigstore: Software signing for everybody. In *ACM CCS*, 2022. DOI: 10.1145/3548606.3560596.
- [14] A. Rundgren. RFC 8785: JSON Canonicalization Scheme (JCS). IETF, 2020.
- [15] B. Laurie, E. Kasper, E. Messeri, and R. Stradling. RFC 9162: Certificate Transparency Version 2.0. IETF, 2021.
- [16] SLSA contributors. *SLSA Specification v1.0*, 2023.
- [17] SPDX Workgroup. *SPDX Specification v3.0.1*, 2024.
- [18] H. Birkholz et al. *An Architecture for Trustworthy and Transparent Digital Supply Chains*. IETF Internet-Draft draft-ietf-scitt-architecture, 2025.
- [19] H. Birkholz et al. *SCITT Reference APIs (SCRAPI)*. IETF Internet-Draft draft-ietf-scitt-scrapi, 2026.
- [20] Anonymous. Trace Interfaces for Anonymous DHTs: Observation Tiers, Committee-Contact Traces, and Exposure Normal Forms. Synthesis 3, The-One-True-Archive (2026).
- [21] Anonymous. Receipt Line Items for Anonymity Claims: A Minimal Tuple Schema for Published Budgets. Series synthesis note (Synthesis 12), The-One-True-Archive (2026).
- [22] Anonymous. *Transparency Primitives for Proof-Carrying Anonymity Claims: Beacons, VRFs, and Append-Only Logs*. Series synthesis note (Synthesis 9), The-One-True-Archive (2026).